Probabilistic Machine Learning

Lecture 04: Discriminative Models: Linear and Logistic Regression

Prof. Dr. Michael Granitzer

Speaker notes

Classical Linear Regression covers the residual sum of squares, the closed form, R^2, basis functions, the LMS update, gradient descent and the Gauss–Markov theorem. Probabilistic Linear Regression shows that squared error is the negative log-likelihood of Gaussian noise of fixed width, so the least-squares fit is the maximum-likelihood estimate (keep the peak, Lecture 03), and that $\hat{\sigma}^2_{\mathrm{ML}} = \mathrm{RSS}/n$ is biased low. Logistic Regression replaces the Gaussian by a Bernoulli: logistic regression, cross-entropy as negative log-likelihood, and no finite maximum on separable data. Both models are discriminative — shortcut 3, model only $p(y \mid x)$. Generative vs. Discriminative Models shows that for yes/no attributes naive Bayes (Lecture 03) has the same linear log-odds and differs only in how the weights are fitted. Calibration answers when a score is a probability (Lecture 01): calibration, what maximum likelihood guarantees, the correction for rebalanced training, and penalties as priors.

Running examples: house prices for 20 flats of 40–120 m² and the 600 m² villa of Lecture 01 (regression); fraud with a base rate of 1 in 1,000 and the yes/no flags of Lecture 03 (classification).

Where We Left Off

Lecture 03 built a classifier from probability and took two shortcuts on the way:

Shortcut	Instead of the ideal	What is dropped
Keep the peak	the most probable parameter (MLE, MAP)	the spread: how sure we are
Assume it apart	factorise the joint — naive Bayes	the dependences between attributes

Today we meet the methods you know from earlier courses — linear and logistic regression — and read them as **probability models**. They never model the inputs:

$$p(y \mid x) \quad \text{instead of} \quad p(x, y) = p(y) \cdot p(x \mid y)$$

SHORTCUT 3

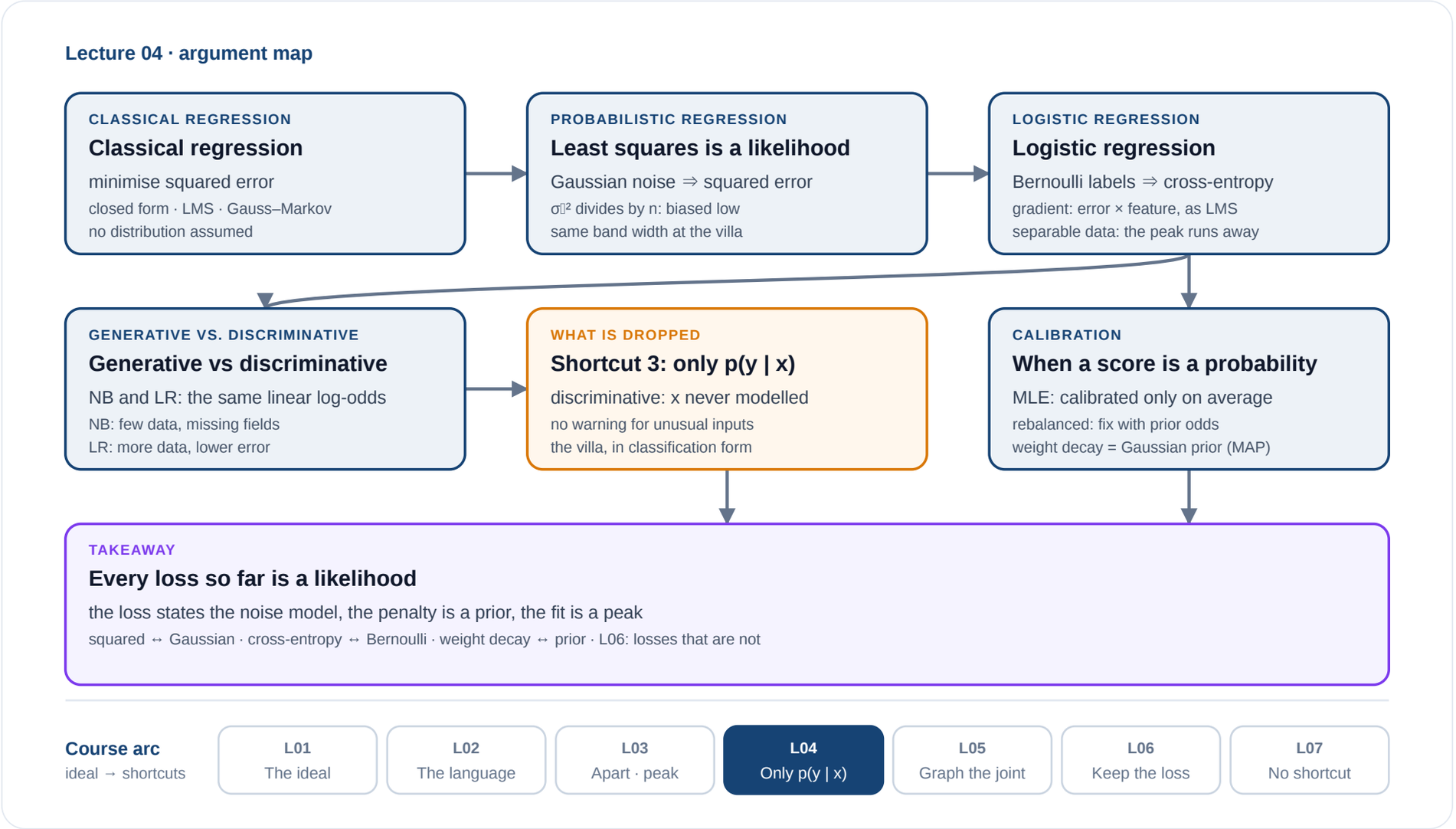
Only what you need: model the label given the inputs — *discriminative*.

Speaker notes

Naive Bayes models the joint $p(x, y) = p(y)\,p(x \mid y)$ and makes it tractable by assuming the attributes independent given the class. A discriminative model avoids the problem in another way: it parametrises $p(y \mid x)$ directly, so no distribution over the inputs is needed and no assumption about their dependences is made. The price is that the model says nothing about x itself — how typical an input is, how to average over a missing attribute, how to generate an example.

Both routes target the same quantity, $p(y \mid x) = p(y)\,p(x \mid y)/p(x)$. The generative route computes it from the joint by Bayes' theorem; the discriminative route fits it as a function of x. Linear regression is discriminative in the same sense: it models the price given the floor area and never the distribution of floor areas. Running examples: fraud with a base rate of 1 in 1,000 and a few yes/no red flags, and the house prices (flats of 40–120 m², the 600 m² villa) of Lecture 01.

Argument Map



Speaker notes

Boxes of the map. Classical Linear Regression: squared error, closed form, LMS and Gauss–Markov, with no distribution assumed. Probabilistic Linear Regression: Gaussian noise implies squared error; $\hat{\sigma}^2$ divides by n and is biased low; the plug-in band is as narrow at the villa as at a flat. Logistic Regression: Bernoulli labels imply cross-entropy; the gradient is error × feature as in LMS; on separable data the maximum-likelihood estimate does not exist. Generative vs. Discriminative Models: naive Bayes and logistic regression share the linear log-odds; naive Bayes is better with few data and handles missing fields, logistic regression reaches lower error with more data. "What is dropped": the inputs are never modelled, so unusual inputs raise no warning. Calibration: maximum likelihood is calibrated only on average, rebalanced scores are repaired with prior odds, weight decay is a Gaussian prior (MAP).

Takeaway box: the loss states the noise model, the penalty is a prior, the fit is a peak. Losses that are not likelihoods follow in Lecture 06. The strip places the lecture in the course arc from the ideal to the shortcuts: this lecture contributes shortcut 3, only $p(y \mid x)$.

Classical Linear Regression

Guiding Question: You have fitted lines by least squares many times. What exactly is the method, and what justifies it?

Speaker notes

The units of this module are shared with the Data Science course and give the classical, distribution-free account. Two of its results return later in the lecture. The LMS update $w_i \leftarrow w_i + \eta e x_i$ (error × feature) has the same form as the gradient of the logistic log-likelihood, with the prediction passed through a sigmoid. The Gauss–Markov theorem justifies least squares under weak conditions — zero-mean, uncorrelated errors of equal variance, no normality — but yields only a point estimate. The probabilistic account of Probabilistic Linear Regression assumes Gaussian noise and in exchange gives a full distribution $p(y \mid x)$: a noise-variance estimate, a predictive band, and assumptions that can be checked on the residuals.

Classification versus Regression

Classification

Y is a finite set of labels.

Will this customer default — yes or no?

Regression

$Y = \mathbb{R}$, the target is a quantity.

What credit line should this customer receive?

The learning machinery is the same; only the shape of Y — and therefore the loss — changes.

Speaker notes

The distinction is fixed by the target set Y, not by the inputs: the same customer record supports a classification task (default, yes or no) and a regression task (credit line in euros). The usual losses follow from Y: 0/1 error or cross-entropy for a finite Y, squared or absolute error for $Y = \mathbb{R}$. Ordinal targets (ratings 1–5) and counts lie in between; treating them as unordered classes ignores their order, treating them as real numbers ignores their discreteness.

The Model

Linear regression predicts the target as a weighted sum of the features:

$$h_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + \ldots + w_p x_p = \mathbf{w}^\top \mathbf{x}$$

with $x_0 = 1$ absorbing the intercept.

The hypothesis space is $\mathbb{R}^{p+1}$: one hypothesis per weight vector.

Speaker notes

Absorbing the intercept means prepending a constant feature $x_0 = 1$ to every input, so that w_0 is handled like any other weight and the design matrix $\mathbf{X}$ has a column of ones. The hypothesis space $\{h_{\mathbf{w}} : \mathbf{w} \in \mathbb{R}^{p+1}\}$ is continuous and finite-dimensional, so the best hypothesis is found by calculus — setting the gradient of the objective to zero — rather than by enumeration.

A weight w_j is the change of the prediction per unit change of x_j with all other features held fixed. Its size depends on the unit of x_j: weights of features on different scales are not directly comparable, and a large weight does not by itself mean an important feature.

The Objective: Residual Sum of Squares

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}\in\mathbb{R}^{p+1}} \mathrm{RSS}(\mathbf{w}), \qquad \mathrm{RSS}(\mathbf{w}) = \sum_{i=1}^{n} \left(y_i - \mathbf{w}^\top \mathbf{x}_i\right)^2$$

RSS measures **effectiveness** — how good the predictions are. Do not confuse it with **efficiency**, which is about runtime and computational effort.

Speaker notes

RSS/n is the mean squared error and $\sqrt{\mathrm{RSS}/n}$ the root-mean-square error, which is in the unit of the target. Squaring makes the objective differentiable everywhere and convex in $\mathbf{w}$ — a quadratic with a unique minimum when $\mathbf{X}^\top\mathbf{X}$ is invertible — and weights large residuals disproportionately: a residual twice as large costs four times as much, so a single outlier can move the fit considerably.

Effectiveness and efficiency are independent: a model can be accurate and slow to fit, or fast and inaccurate. Calling an accurate model "efficient" is a common misuse of the term.

One Feature: The Closed Form

For a single feature the minimiser can be written down directly:

Slope

$$\hat{w}_1 = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sum_i (x_i - \overline{x})^2}$$

The covariance of x and y over the variance of x.

Intercept

$$\hat{w}_0 = \overline{y} - \hat{w}_1 \overline{x}$$

The fitted line passes through the point of means.

Speaker notes

Derivation. Setting $\partial \mathrm{RSS}/\partial w_0 = -2\sum_i (y_i - w_0 - w_1 x_i) = 0$ gives $w_0 = \overline{y} - w_1 \overline{x}$. Substituting this into $\partial \mathrm{RSS}/\partial w_1 = -2\sum_i x_i (y_i - w_0 - w_1 x_i) = 0$ gives the slope formula. Dividing numerator and denominator by $n - 1$ shows $\hat{w}_1 = \mathrm{cov}(x, y)/\mathrm{var}(x) = r_{xy}\, s_y / s_x$, with r_{xy} the correlation coefficient; the slope is undefined when all x_i are equal.

Consequences: the residuals sum to zero, and the line passes through $(\overline{x}, \overline{y})$. Regressing x on y gives a different line, because least squares measures errors vertically, in y only; the two lines coincide only when $|r_{xy}| = 1$.

How Much Variance Is Explained?

$$\mathrm{TSS} = \mathrm{ESS} + \mathrm{RSS}, \qquad R^2 = \frac{\mathrm{ESS}}{\mathrm{TSS}} = 1 - \frac{\mathrm{RSS}}{\mathrm{TSS}}$$

where TSS is the total, ESS the explained and RSS the residual sum of squares (the split holds for a least-squares fit with an intercept).

R^2 is the fraction of the target's variation the model accounts for: 0 means no better than predicting the mean, 1 means an exact fit.

Training R^2 never decreases when a feature is added, however useless the feature. A high training R^2 is not evidence that a model generalises — test R^2 and adjusted R^2 can and do fall.

Speaker notes

Definitions: $\mathrm{TSS} = \sum_i (y_i - \overline{y})^2$, $\mathrm{ESS} = \sum_i (\hat{y}_i - \overline{y})^2$, $\mathrm{RSS} = \sum_i (y_i - \hat{y}_i)^2$. The split holds because, with an intercept, the residuals sum to zero and are orthogonal to the fitted values, so the cross term vanishes. For a single feature, $R^2 = r_{xy}^2$.

Without an intercept, or on test data, $1 - \mathrm{RSS}/\mathrm{TSS}$ can be negative: the model is then worse than predicting the mean. Training R^2 cannot fall when a feature is added, because the larger model contains the smaller one (set the new weight to 0), so its minimal RSS is at most as large. Adjusted $R^2 = 1 - (1 - R^2)\,\frac{n-1}{n-p-1}$ charges for each of the p features and falls when a feature reduces RSS by less than a random feature would on average.

Higher Dimensions

With p features the same objective in matrix form,

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

gives the optimum in one step whenever $\mathbf{X}^\top \mathbf{X}$ is invertible.

> Invertibility fails when features are linearly dependent or when $p > n$ — more unknowns than equations. Both are common in practice; regularisation is the standard remedy.

Speaker notes

Derivation: $\mathrm{RSS}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$ has gradient $-2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$; setting it to zero gives the normal equations $\mathbf{X}^\top\mathbf{X}\,\mathbf{w} = \mathbf{X}^\top\mathbf{y}$, with $\mathbf{X}$ of size $n \times (p+1)$ including the column of ones. $\mathbf{X}^\top\mathbf{X}$ is invertible exactly when the columns of $\mathbf{X}$ are linearly independent, which requires $n \geq p+1$.

When it is singular the normal equations still have solutions, but infinitely many; the pseudo-inverse selects the one of minimal norm. Strongly correlated features leave the matrix invertible but ill-conditioned, and the weights become unstable. Numerical software solves the system by QR or Cholesky decomposition instead of forming the inverse. Ridge regression replaces $\mathbf{X}^\top\mathbf{X}$ by $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$, which is invertible for every $\lambda > 0$. $p > n$ is the normal situation in genomics and text, where features outnumber examples.

Takeaway

Linear regression fits a weighted sum by minimising squared residuals. Its solution is built from **covariance over variance**, and R^2 reports how much variation it explains — not whether it will generalise.

Linear in the Parameters, Not in the Features

A model is **linear** when it is linear in the **weights** $\mathbf{w}$. The features may be arbitrary transformations of the input.

Plain

$$h(\mathbf{x}) = w_0 + w_1 x$$

a straight line

Polynomial

$$h(\mathbf{x}) = w_0 + w_1 x + w_2 x^2$$

a curve — still a *linear model*

Basis functions

$$h(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x})$$

any fixed feature map ϕ

What *linear* forbids: a transformation that has to be learned from the data. The feature map is fixed before fitting; only the weights are learned.

Speaker notes

Linearity in the weights is what makes RSS a quadratic function of $\mathbf{w}$ and keeps the normal equations valid: with a feature map ϕ, the design matrix simply has entries $\Phi_{ij} = \phi_j(\mathbf{x}_i)$. Fixed maps include polynomials, interactions $x_1 x_2$, $\log x$, indicator variables for categories, periodic terms $\sin(2\pi t/T)$ with known period T, and radial basis functions with fixed centres.

Not expressible in this way: models whose transformation has parameters fitted to the data, such as $w_1 \sin(w_2 x)$ with a learned frequency w_2, radial basis functions with learned centres, or the hidden layers of a neural network. Such models are non-linear in their parameters; their losses are in general non-convex and have no closed-form minimiser.

Consequences

The good

The closed-form solution and all statistical guarantees carry over unchanged.

The catch

The feature map is a **modelling decision**. Choose too rich a basis and the model overfits; too poor and it underfits.

> Increasing the polynomial degree turns a dial on the bias–variance axis.

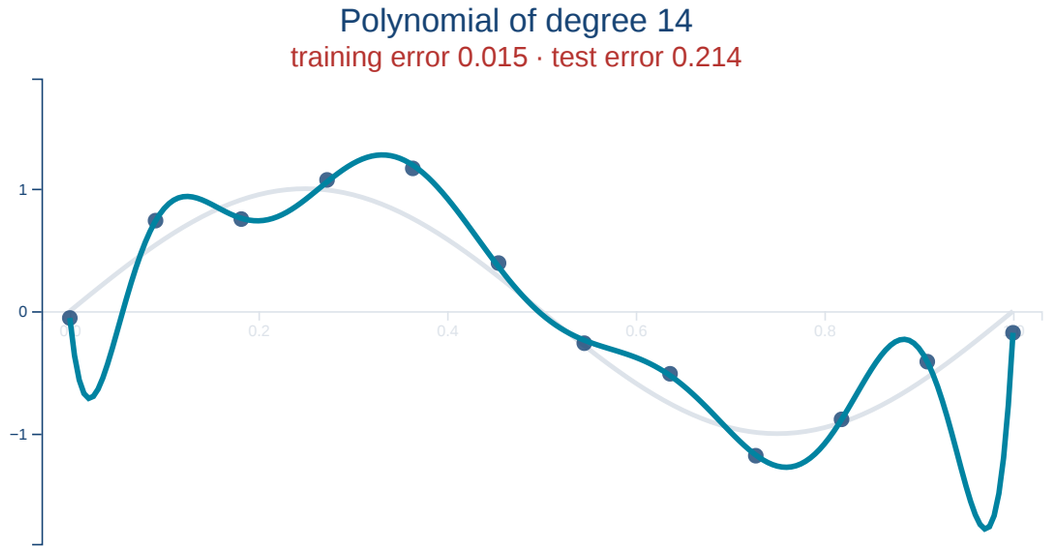
Speaker notes

What carries over is everything that holds for a linear model in $\phi(\mathbf{x})$: the closed form, the projection geometry, and the Gauss–Markov guarantee, provided the model is correctly specified in the chosen basis. Choosing the basis is choosing the hypothesis space — the inductive bias stated explicitly.

Too rich a basis has low bias and high variance; with as many basis functions as data points the fit interpolates the training data. Too poor a basis has high bias: no amount of data lets a straight line fit a sine. Raw polynomial features $x, x^2, \ldots, x^k$ are strongly correlated and make $\mathbf{X}^\top\mathbf{X}$ ill-conditioned at high degree; rescaling x to $[-1, 1]$ or using orthogonal polynomials mitigates this.

Turning the Capacity Dial

Linear in the parameters, any basis in the features: a polynomial of degree k is still "linear regression" — and k is a dial.



Speaker notes

A polynomial of degree k is linear regression on the basis $(1, x, \ldots, x^k)$, so the closed form applies at every degree. The visualisation fits 12 noisy samples of $\sin(2\pi x)$ (noise standard deviation 0.18) and reports the root-mean-square error on these points and on a second sample at the same inputs with fresh noise. For degrees 1, 2, 3, 5, 9, 14 the training error is 0.551, 0.546, 0.198, 0.150, 0.086, 0.015 and the test error 0.487, 0.492, 0.171, 0.162, 0.206, 0.214.

Training error never rises with the degree, because each model contains the previous one. Test error is lowest at degree 5 and cannot fall much below the noise level 0.18. Degree 14 has 15 coefficients for 12 points: $\mathbf{X}^\top\mathbf{X}$ is singular and infinitely many polynomials pass (almost) through the data. Because the test inputs coincide with the training inputs, the rise in test error understates the overfitting; between the data points the high-degree curves swing far from the sine. Each degree is fitted by keeping the peak of a Gaussian likelihood. A penalty $\lambda\|\mathbf{w}\|^2$ — a Gaussian prior on $\mathbf{w}$ — reduces capacity without lowering the degree.

Fitting h by Repeated Correction

LMS (least mean squares)

1. Initialise the weights w.
2. Pick a training example $(\mathbf{x}, y)$ from the training set D.
3. Compute the prediction $h_w(\mathbf{x})$ and the error $e = y - h_w(\mathbf{x})$.
4. Update every weight: $w_i \leftarrow w_i + \eta e x_i$.
5. Repeat 2–4 until the error on D stops decreasing.

Speaker notes

The update is a gradient step on the squared error of a single example. With $e = y - \mathbf{w}^\top \mathbf{x}$ and $E = \frac{1}{2} e^2$, $\partial E / \partial w_i = -e\, x_i$, so $w_i \leftarrow w_i - \eta\, \partial E / \partial w_i = w_i + \eta e x_i$. Each weight moves in the direction that reduces this example's error, in proportion to its feature value. The rule is also called the Widrow–Hoff or delta rule.

Examples are usually visited in random order; one pass over D is an epoch. With a constant η the weights do not settle exactly at the least-squares solution but keep fluctuating around it, because every step follows a single noisy example; a decreasing learning rate or averaging of the iterates makes them converge.

What the Update Rule Encodes

Direction

The sign of e decides whether the prediction goes up or down.

Attribution

The factor x_i gives each feature credit proportional to its value.

Step size

The learning rate η trades speed against stability.

Too large an η oscillates or diverges; too small an η converges slowly. It is a hyperparameter — chosen, not learned.

Speaker notes

Direction: if $e > 0$ the prediction was too low, and every weight with $x_i > 0$ increases. Attribution: a feature with $x_i = 0$ receives no update from that example, and features on large numeric scales receive large updates — one reason features are standardised before gradient-based fitting.

Step size: one update changes this example's prediction by $\eta e \|\mathbf{x}\|^2$, so the new error is $e\,(1 - \eta\|\mathbf{x}\|^2)$. The update overshoots the target when $\eta\|\mathbf{x}\|^2 > 1$ and enlarges the error when $\eta\|\mathbf{x}\|^2 > 2$; normalised LMS divides the step by $\|\mathbf{x}\|^2$ for this reason. The learning rate does not appear in the training loss and cannot be fitted by minimising it; it is chosen by comparing runs, typically on validation data.

Two Ways to the Same Minimum

Direct (closed form)

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Exact, one step. Requires forming and inverting a $(p+1) \times (p+1)$ matrix — costly for large p, impossible when it is singular.

Iterative (gradient descent)

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \mathrm{RSS}(\mathbf{w})$$

Approximate, many steps. Scales to large p and n, works when the direct solution does not exist, and extends to models with no closed form at all.

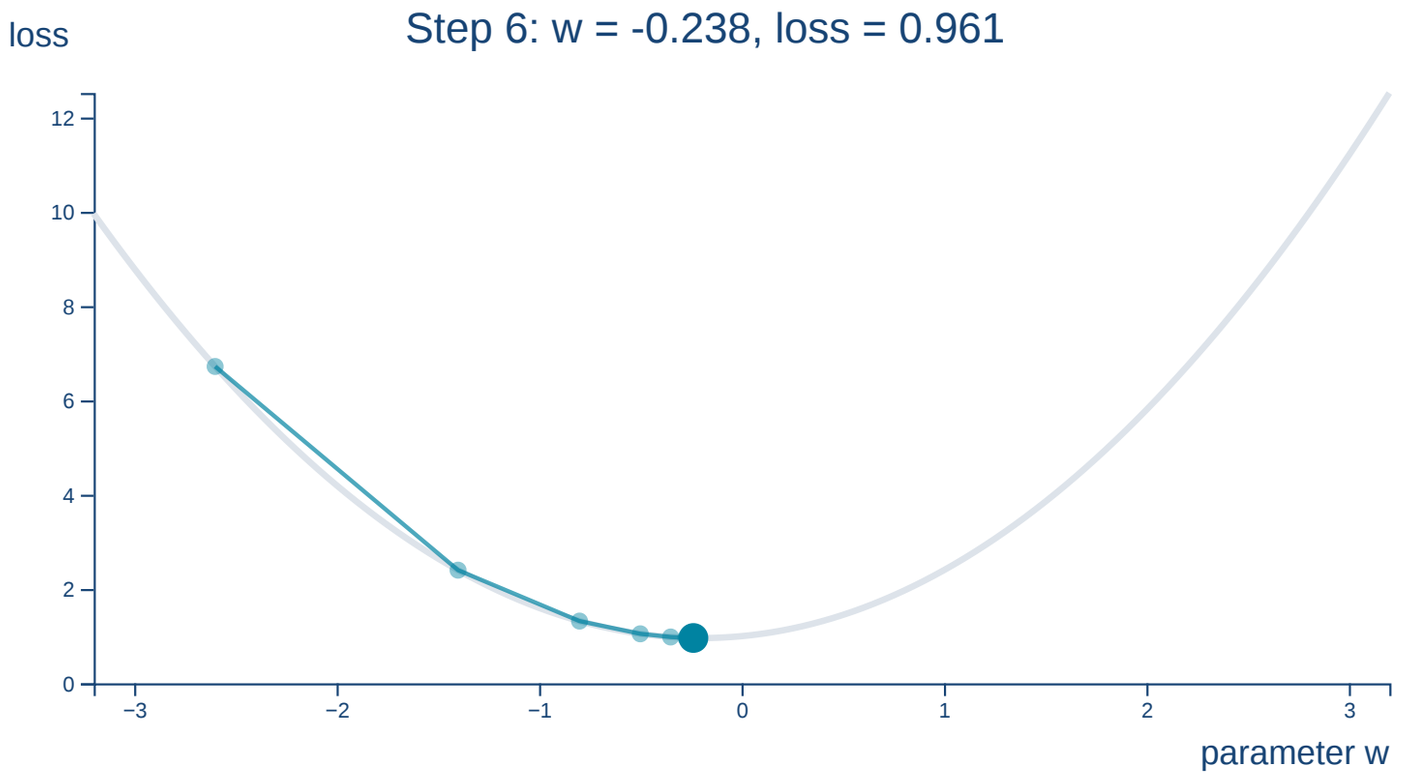
The iterative rule is exactly the **LMS algorithm**: its update $w_i \leftarrow w_i + \eta e x_i$ *is* a gradient step on RSS, taken per example instead of per dataset (the factor 2 is absorbed into η).

Speaker notes

Costs: forming $\mathbf{X}^\top\mathbf{X}$ takes $O(np^2)$ operations and solving the $(p+1)\times(p+1)$ system $O(p^3)$; one full gradient step takes $O(np)$. The gradient is $\nabla\mathrm{RSS}(\mathbf{w}) = -2\mathbf{X}^\top(\mathbf{y}-\mathbf{X}\mathbf{w})$: a full-batch step sums error × feature over all examples, while LMS uses one example per update (stochastic gradient descent).

Because RSS is convex, gradient descent with a small enough learning rate approaches the minimum that the closed form computes. When $\mathbf{X}^\top\mathbf{X}$ is singular the minimum is not unique; started at $\mathbf{w}=0$, gradient descent converges to the minimum-norm solution. Logistic regression and neural networks have no closed-form minimiser, so the iterative method is the general one and the closed form the special case.

Watching Gradient Descent Work

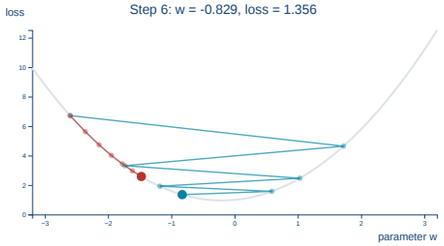


Speaker notes

The curve is a one-parameter stand-in for RSS, $L(w) = w^2 + 0.4w + 1$, with its minimum at $w = -0.2$. A step is $w \leftarrow w - \eta(2w + 0.4)$, so the distance to the minimum is multiplied by $1 - 2\eta$ in every step. With $\eta = 0.25$ the factor is 0.5: starting at $w = -2.6$ (distance 2.4), the distance halves with each step and is 0.0375 after six steps ($w \approx -0.24$).

The steps shrink although η is constant: the step length is $\eta|L'(w)|$, and the gradient is proportional to the distance from the minimum. Steps are therefore largest far from the minimum, where the slope is steepest.

The Learning Rate Decides Everything



Teal: $\eta = 0.9$ — overshoots and oscillates across the minimum. Red: $\eta = 0.05$ — stable, but slow: after six steps it has covered about half the distance.

> The learning rate is a **hyperparameter**: it is chosen, not learned, and both failure modes are silent — one looks unstable, the other looks converged.

Speaker notes

On the same loss the distance to the minimum is multiplied by $1 - 2\eta$ per step. For $\eta = 0.9$ the factor is -0.8: every step jumps across the minimum and removes only 20% of the remaining distance, so after six steps $0.8^6 \approx 26\%$ of it remains. For $\eta = 0.05$ the factor is 0.9, leaving $0.9^6 \approx 53\%$ — the "about half" on the slide. The iteration diverges for $\eta > 1$, where $|1 - 2\eta| > 1$, and $\eta = 0.5$ reaches the minimum in a single step.

In general, gradient descent on a quadratic with Hessian $\mathbf{H}$ is stable only for $\eta < 2/\lambda_{\max}(\mathbf{H})$; here $\mathbf{H} = 2$. The slow run is the harder failure to detect: over a short window its loss curve flattens like that of a converged run, whereas the oscillation of a too-large η is visible in the parameter trace.

Takeaway

> Use the **closed form** when the matrix is small and well-conditioned; use **gradient descent** when it is not — and for most models beyond linear regression, no closed form exists at all.

The Question Nobody Asks

We minimised the *squared* residuals. Why squares — why not absolute deviations, or fourth powers?

Speaker notes

Squared error has several independent justifications. It is differentiable and gives a closed form. Its minimiser is a mean: the constant minimising expected squared deviation is the mean, the one minimising expected absolute deviation is the median, so least squares estimates the conditional mean of y. Under the conditions of the Gauss–Markov theorem it gives the linear unbiased estimator of smallest variance, and under Gaussian noise it is the maximum-likelihood criterion. Absolute deviations give least-absolute-deviation regression, which estimates the conditional median and is less sensitive to outliers; higher even powers weight large residuals even more strongly and make the fit more sensitive to them.

Geometric View

The predictions $\mathbf{Xw}$ live in the **column space** of $\mathbf{X}$ — everything the features can express. The observed $\mathbf{y}$ generally does not.

Least squares picks the point of that space **closest to $\mathbf{y}$ in Euclidean distance**: the orthogonal projection. The residual is perpendicular to every feature.

Consequence

The residual carries no remaining linear information about the features.

Speaker notes

The normal equations $\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}) = \mathbf{0}$ state exactly that the residual vector is orthogonal to every column of $\mathbf{X}$. The fitted values are $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ with the hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$, the orthogonal projection onto the column space. Pythagoras on $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{e}$, after centring, gives TSS = ESS + RSS.

Adding a feature enlarges the column space, so the closest point can only get closer and RSS cannot increase. "No remaining linear information" means that the residuals are uncorrelated with each feature in the sample; a non-linear pattern of the residuals against a feature remains possible and indicates a misspecified model.

The Gauss–Markov Theorem

Gauss–Markov (informal). Among all estimators that are

- **linear** in the observed outputs $(y_1, \ldots, y_n)$, and
- **unbiased**,

the least-squares estimator has the **lowest variance**. It is the *Best Linear Unbiased Estimator* — BLUE.

Best

lowest variance among the candidates

Linear

a linear function of the observations

Unbiased

right on average over repeated samples

Speaker notes

Formal statement: for $\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\varepsilon}$ with $\mathbf{X}$ fixed and of full column rank, $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$ and $\mathrm{Cov}(\boldsymbol{\varepsilon}) = \sigma^2\mathbf{I}$, every linear unbiased estimator $\tilde{\mathbf{w}} = \mathbf{C}\mathbf{y}$ has a covariance exceeding $\mathrm{Cov}(\hat{\mathbf{w}}_{\mathrm{OLS}}) = \sigma^2(\mathbf{X}^\top\mathbf{X})^{-1}$ by a positive semi-definite matrix. Hence every linear combination $\mathbf{c}^\top\mathbf{w}$, including each single weight and each prediction, is estimated with the smallest variance by OLS.

The theorem says nothing about non-linear or biased estimators. Mean squared error is $\mathrm{bias}^2 + \mathrm{variance}$, so a biased estimator can be better: for ridge regression there always exists a $\lambda > 0$ whose weights have a smaller mean squared error than OLS (Hoerl & Kennard, 1970). This does not contradict the theorem, which ranks only unbiased linear estimators.

What This Buys, and What It Costs

The payoff

Minimising RSS yields optimal statistical properties under **mild assumptions** — no normality needed, only a correctly specified linear model, linearly independent features, and zero-mean, uncorrelated errors of constant variance.

The price

Alternatives — absolute error, Huber loss — are **more robust to outliers** but give up the BLUE guarantee.

Squared error is not an arbitrary convenience: it is the loss whose minimiser is the best linear unbiased estimator. Departing from it is legitimate — but it is a trade, and you should be able to name what you traded.

Speaker notes

The assumptions in full: the model is linear in the parameters and correctly specified; the features are linearly independent; the errors have mean zero, equal variance and are uncorrelated. Normality is not required for BLUE; it is required for exact t- and F-tests and intervals in small samples. With unequal variances OLS remains unbiased but is no longer best; weighted least squares with weights $1/\sigma_i^2$ restores optimality.

Robust losses trade efficiency under ideal conditions for stability under outliers. The Huber loss is quadratic for small residuals and linear for large ones; absolute error is linear throughout. Their minimisers are not linear in $\mathbf{y}$, so the theorem does not cover them. The same trade appears in descriptive statistics: the mean minimises squared deviations and is pulled by outliers, the median minimises absolute deviations and is not.

Probabilistic Linear Regression

Guiding Question: You have minimised squared error many times. What did that choice assume about the world — and what does it leave out?

The Same Model, Told Differently

Classical account

Choose the line that **minimises squared error**:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

Probabilistic account

State how a price **arises** for a given flat:

$$y = \mathbf{w}^\top \mathbf{x} + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

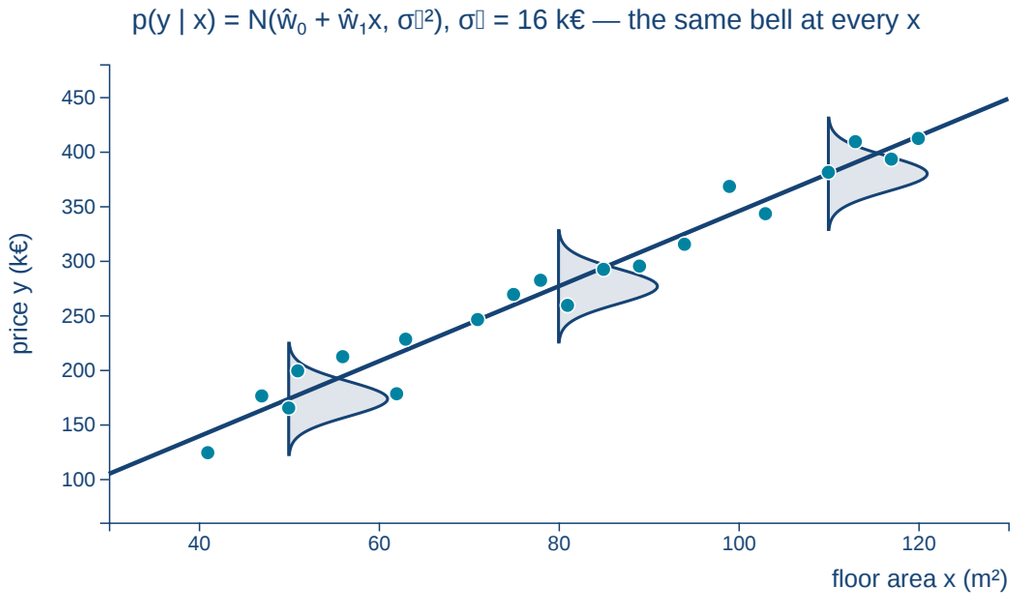
then keep the peak of the likelihood.

Do the two accounts give the same $\hat{\mathbf{w}}$ — and if so, what did the classical account assume without saying it?

Speaker notes

The two accounts give the same $\hat{\mathbf{w}}$: the Gaussian log-likelihood is a constant minus $\mathrm{RSS}/(2\sigma^2)$ (derivation below). What the classical account assumed without saying it is a noise model: errors that are Gaussian, of the same width at every x, and independent between rows. The probabilistic account states these assumptions as a model of how y arises given $\mathbf{x}$, and returns more than $\hat{\mathbf{w}}$: an estimate of σ^2, a predictive distribution for a new y, and assumptions that can be checked on the residuals. Gauss–Markov justifies the same $\hat{\mathbf{w}}$ under weaker conditions (no normality) but provides no distribution for y.

A Distribution at Every x



A **conditional distribution** $p(y \mid x) = \mathcal{N}(\mathbf{w}^\top \mathbf{x}, \sigma^2)$: one bell per x, **equally wide**. Nothing about how x arises.

Speaker notes

Data: 20 flats of 41–120 m². Fit: $\hat{y} \approx 1.4 + 3.44\,x$ (k€, m²), with $\hat{\sigma}_{\mathrm{ML}} \approx 16.0$ k€. Step 1 shows the fitted line, step 2 Gaussian bells at 50, 80 and 110 m², all with standard deviation $\hat{\sigma}$. Reading one bell: an 80 m² flat has a predicted mean of about 276 k€, and 95% of such prices lie within $\pm 1.96\,\hat{\sigma} \approx \pm 31$ k€.

The line is the conditional mean $\mathbb{E}[y \mid x]$; the bell is the conditional distribution, and the loss is the exponent of the bell (next slide). The model contains no distribution over x: it never represents which floor areas are typical, so it cannot recognise that a 600 m² villa lies far outside its data. This is shortcut 3, model only $p(y \mid x)$, named in Lecture 04.

The Derivation

With $p(y_i \mid \mathbf{x}_i, \mathbf{w}) = \mathcal{N}(y_i \mid \mathbf{w}^\top \mathbf{x}_i, \sigma^2)$ and i.i.d. rows:

$$\ell(\mathbf{w}) = \sum_{i=1}^{n} \log \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mathbf{w}^\top \mathbf{x}_i)^2}{2\sigma^2}\right) = \underbrace{-\frac{n}{2}\log(2\pi\sigma^2)}_{\text{no } \mathbf{w}} - \frac{1}{2\sigma^2}\mathrm{RSS}(\mathbf{w})$$

> Maximising the Gaussian log-likelihood **is** minimising RSS: $\hat{\mathbf{w}}_{\mathrm{ML}} = \hat{\mathbf{w}}_{\mathrm{OLS}}$. Squared error is the exponent of the bell.

Speaker notes

The first term does not depend on $\mathbf{w}$ and $1/(2\sigma^2) > 0$, so for every fixed σ^2 the arg max of ℓ over $\mathbf{w}$ is the arg min of RSS. The value of σ^2 therefore does not affect $\hat{\mathbf{w}}$; it is estimated afterwards from the residuals. The i.i.d. assumption is what turns the joint likelihood into a product and the log-likelihood into a sum over rows.

The equivalence is specific to Gaussian noise with one shared variance. With observation-specific variances σ_i^2 the likelihood leads to weighted least squares, $\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 / \sigma_i^2$; with Laplace noise it leads to least absolute deviations.

What Squared Error Assumed

the noise model says	squared error silently assumes	check on the residuals
ε has mean 0	the line is the right shape	no pattern against x
one σ for every x	**homoscedastic** noise	no fan shape
rows independent	i.i.d. (Lecture 02)	no pattern in time or space
ε Gaussian	thin tails: large errors are rare	no heavy tails, few outliers

Choosing a loss and choosing a noise model are **one decision**. Squared error ⇔ Gaussian noise; absolute error ⇔ Laplace noise — heavier tails, so a large residual is less surprising and an outlier pulls the fit less.

Speaker notes

Each row turns a property of the loss into a checkable statement about the data. Residuals plotted against the fitted values or against each feature should show no trend (mean 0, right shape) and constant spread (homoscedasticity); residuals in collection order or by location should show no correlation (independence); a normal Q–Q plot of the residuals reveals heavy tails.

Laplace row: the density $\frac{1}{2b}e^{-|\varepsilon|/b}$ gives $-\log p = |\varepsilon|/b + \log(2b)$, so its maximum-likelihood fit minimises the sum of absolute residuals. The derivative with respect to a residual is bounded ($\pm 1/b$), whereas for the Gaussian it grows linearly with the residual — the reason an outlier pulls an absolute-error fit less. A violated row does not always spoil $\hat{\mathbf{w}}$: with non-Gaussian or unequal-variance noise of mean zero, least squares is still unbiased; the intervals fail first.

The Peak of σ^2

The same likelihood also has a peak in σ^2:

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{\mathrm{RSS}}{2\sigma^4} = 0 \quad \Longrightarrow \quad \hat{\sigma}^2_{\mathrm{ML}} = \frac{\mathrm{RSS}}{n}$$

Which of Lecture 02's two variances is this — and is it too small or too large?

Speaker notes

Answer: the $1/n$ variance, which is biased low. The residuals come from a line fitted to the same data, and the fit absorbs part of the noise. For p fitted weights the residual vector lies in an $(n-p)$-dimensional subspace, so $\mathbb{E}[\mathrm{RSS}] = (n-p)\sigma^2$ and $\mathbb{E}[\hat{\sigma}^2_{\mathrm{ML}}] = \frac{n-p}{n}\sigma^2$. The $n-1$ of Lecture 02 is the case $p=1$, a single estimated mean.

Houses: $5{,}097/20 \approx 254.8$ gives $\hat{\sigma} \approx 16.0$ k€; $5{,}097/18 \approx 283.2$ gives 16.8 k€. The bias is 10% of the variance for $n=20$, $p=2$, and 40% for $n=5$, $p=2$. It always points the same way: taking the peak in σ^2 jointly with $\mathbf{w}$ (Lecture 03) makes the model look less noisy than the process that generated the data.

The Peak of σ^2

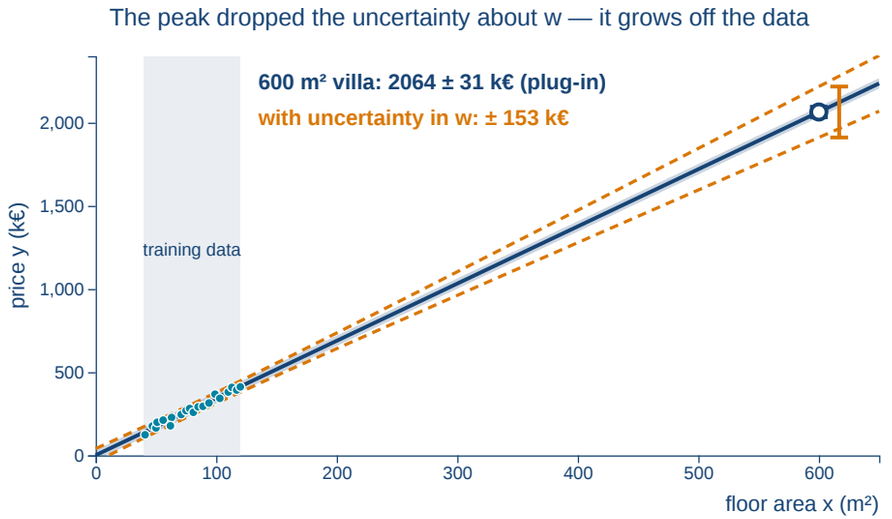
The same likelihood also has a peak in σ^2:

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{\mathrm{RSS}}{2\sigma^4} = 0 \quad \Longrightarrow \quad \hat{\sigma}^2_{\mathrm{ML}} = \frac{\mathrm{RSS}}{n}$$

Which of Lecture 02's two variances is this — and is it too small or too large?

> The $1/n$ one. The line was fitted to the same data, so residuals are smaller than the true errors: $\hat{\sigma}^2_{\mathrm{ML}}$ is **biased low**. The unbiased version divides by $n - p$ for p fitted weights. Houses: $n = 20$, $p = 2$, RSS $\approx 5{,}097$ $\rightarrow$ $\hat{\sigma} \approx 16.0$ instead of 16.8 k€.

The Villa Again



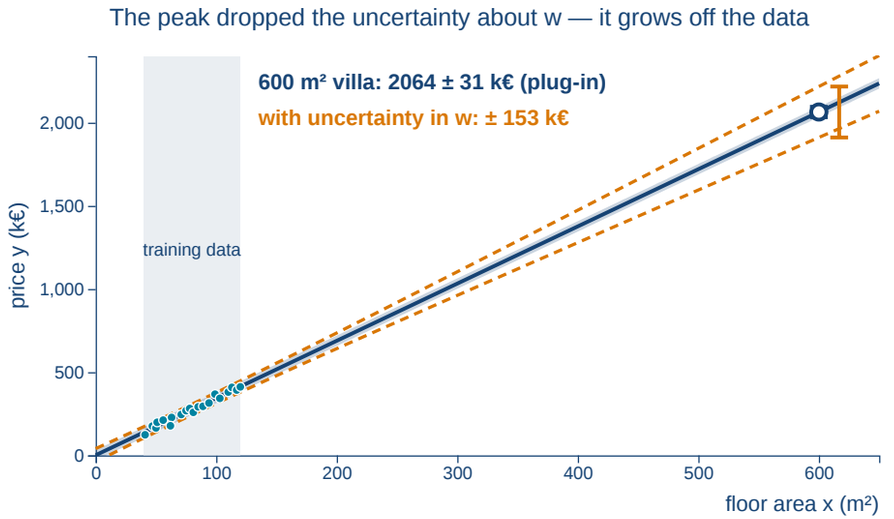
The band is as narrow at the villa as at a flat. Is the model as sure?

Speaker notes

Step 0: the fit and the plug-in band $\hat{y} \pm 1.96\,\hat{\sigma} = \pm 31$ k€, which treats $\hat{\mathbf{w}}$ as exact. Step 1: the villa of Lecture 01 at 600 m², predicted at about 2,064 k€ with the same ± 31 k€. Step 2: the band that also counts the uncertainty about the two weights, half-width $1.96\,\hat{\sigma}\sqrt{1 + 1/n + (x - \overline{x})^2 / \sum_i (x_i - \overline{x})^2}$: ± 32 k€ at 80 m², ± 153 k€ at 600 m².

The term $(x - \overline{x})^2$ makes the variance grow quadratically with the distance from the mean floor area. This is epistemic uncertainty, which data near 600 m² would reduce — the spread that keeping the peak drops (Lecture 03). The band here uses $\hat{\sigma}_{\mathrm{ML}}$ and 1.96; the textbook prediction interval uses the unbiased estimate and a t-quantile with $n - 2$ degrees of freedom and is slightly wider. Both assume the straight line holds up to 600 m²; whether villas follow the price per m² of flats is a question about the model that no band computed from flats can answer.

The Villa Again



The band is as narrow at the villa as at a flat. Is the model as sure?

No: ± 31 k€ is only the noise $\hat{\sigma}$. The uncertainty about $\mathbf{w}$ — dropped with the peak — grows off the data: ± 153 k€ at the villa (Lecture 07).

Mini-Exercise

The residuals of your house-price model fan out: about ± 10 k€ for 40 m² flats, ± 60 k€ for 120 m² flats.

Which assumption fails — and which output of the model can you no longer trust?

Speaker notes

A fan shape violates homoscedasticity. The line can still be adequate: $\hat{\mathbf{w}}$ remains unbiased, but it is no longer the minimum-variance linear unbiased estimator, since Gauss–Markov requires equal variances. The single $\hat{\sigma}$ averages the spreads, so the band is too wide for small flats and too narrow for large ones, and the standard errors of the weights are unreliable as well.

Remedies state a different noise model: σ as a function of x (here roughly proportional to the floor area), fitted by weighted least squares with weights $1/\sigma^2(x)$; or a model for $\log y$, where noise proportional to the price becomes additive noise of constant variance.

Mini-Exercise

The residuals of your house-price model fan out: about ± 10 k€ for 40 m² flats, ± 60 k€ for 120 m² flats.

Which assumption fails — and which output of the model can you no longer trust?

One σ for every x (homoscedasticity). The line may still be fine, but the **band** is not: a single $\hat{\sigma}$ is too wide for small flats and too narrow for large ones.

Takeaway

Least squares **is** maximum likelihood under Gaussian noise: the loss states the noise model, and the model is a distribution $p(y \mid x)$. It is also a **peak**: $\hat{\sigma}^2$ divides by n, and the band is as narrow at the villa as at a flat — the uncertainty about $\mathbf{w}$ was dropped.

Logistic Regression

Guiding Question: Lecture 01's fraud model said 0.55. Where does such a number come from — and what does the model refuse to know?

Why Not Regress on 0/1?

Code fraud as $y = 1$, legitimate as $y = 0$, and fit the line of probabilistic linear regression.

Speaker notes

Least squares on 0/1 labels (the linear probability model) fails for two reasons. The loss: a fraud with score 3 has residual -2 and squared error 4, although it is classified correctly; minimising RSS reduces such residuals, so points far on the correct side tilt the boundary. The output: the linear score is unbounded and cannot be read as a probability, and the Gaussian noise model assigns density to every real y although only 0 and 1 occur. A 0/1 label also has variance $h(1-h)$, which depends on $\mathbf{x}$ and contradicts the single σ of the Gaussian model. Replacing the Gaussian by a Bernoulli and modelling the log-odds linearly removes both problems.

Why Not Regress on 0/1?

Code fraud as $y = 1$, legitimate as $y = 0$, and fit the line of probabilistic linear regression.

Squared error punishes being right

A transaction far on the correct side has a large residual, so least squares drags the boundary towards cases it already classifies correctly.

The output is not a probability

$\mathbf{w}^\top \mathbf{x}$ can be 17 or -40. The Gaussian bell of the regression model puts mass on every real y — for a label that is 0 or 1.

Keep the linear score, change what it models: the **log-odds**, and a noise model that produces 0s and 1s.

Log-Odds Are Linear

$$\ln \frac{P(y=1 \mid \mathbf{x})}{1-P(y=1 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x} \qquad \Longleftrightarrow \qquad P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^\top \mathbf{x}}}$$

read the weights as	fraud example
the intercept: the log-odds when every feature is 0	no features at all: $w_0 = \ln \frac{1}{999} \approx -6.9$, the base rate 1 in 1,000
a weight: what one unit of a feature **adds** to the log-odds	"new device" with $w = \ln 6 \approx 1.8$: odds $\times 6$
the boundary: where $\mathbf{w}^\top \mathbf{x} = 0$	score 0.5 — still a straight line

Speaker notes

The sigmoid $\sigma(z) = 1/(1+e^{-z})$ is the inverse of the logit $\ln \frac{p}{1-p}$; the model therefore states that the log-odds are linear in $\mathbf{x}$. Intercept: with no features the maximum-likelihood fit sets $\sigma(w_0)$ to the fraction of positives, $1/1000$, so $w_0 = \ln(1/999) \approx -6.91$ — the estimate h/n of Lecture 03 on the log-odds scale.

Weights: one unit more of x_j adds w_j to the log-odds and multiplies the odds by e^{w_j}. Odds are not probabilities: multiplying the odds by 6 moves P from 0.10% to 0.60% at the base rate, but from 0.5 to 0.86 at odds 1. The boundary $\mathbf{w}^\top \mathbf{x} = 0$ is a hyperplane; another threshold than 0.5 shifts it in parallel. Adding log-odds is the arithmetic of multiplying likelihood ratios in Bayes' theorem (Lecture 02), which naive Bayes uses (Lecture 03).

Where Does Cross-Entropy Come From?

$$L(\mathbf{w}) = -\sum_{i=1}^{n} \Big[y_i \ln h_i + (1 - y_i) \ln(1 - h_i) \Big], \qquad h_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

You have minimised this loss many times. Which noise model does it state?

Speaker notes

Answer: a Bernoulli noise model. Given $\mathbf{x}_i$, the label is a draw $y_i \sim \mathrm{Bernoulli}(h_i)$ with $h_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$; $h_i^{y_i}(1-h_i)^{1-y_i}$ is a compact way of writing h_i for $y_i = 1$ and $1 - h_i$ for $y_i = 0$. Taking logs and summing over i.i.d. rows gives $\ell(\mathbf{w}) = -L(\mathbf{w})$. It is the derivation of the coin estimate h/n in Lecture 03, with the single bias θ replaced by a bias $h(\mathbf{x})$ for every row.

The name: $-[y \ln h + (1-y)\ln(1-h)]$ is the cross-entropy between the observed label distribution (all mass on y) and the predicted one. The model is conditional: it describes y given $\mathbf{x}$ and contains no distribution from which a transaction $\mathbf{x}$ could be generated.

Where Does Cross-Entropy Come From?

$$L(\mathbf{w}) = -\sum_{i=1}^{n} \Big[y_i \ln h_i + (1 - y_i) \ln(1 - h_i) \Big], \qquad h_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

You have minimised this loss many times. Which noise model does it state?

Given $\mathbf{x}$, the label is a **Bernoulli** draw with bias h: $P(y_i \mid \mathbf{x}_i, \mathbf{w}) = h_i^{y_i}(1 - h_i)^{1 - y_i}$. With i.i.d. rows,

$$\ell(\mathbf{w}) = \sum_{i=1}^{n} \Big[y_i \ln h_i + (1 - y_i) \ln(1 - h_i) \Big] = -L(\mathbf{w})$$

> Cross-entropy **is** the Bernoulli negative log-likelihood — Lecture 03's coin, biased by $\mathbf{x}$. How $\mathbf{x}$ arises is never said.

The Same Update Again

With $\sigma' = \sigma(1-\sigma)$, the gradient of the log-likelihood is

$$\frac{\partial \ell}{\partial \mathbf{w}} = \sum_{i=1}^{n} (y_i - h_i)\mathbf{x}_i$$

model	noise	loss	gradient of ℓ
linear regression	Gaussian	squared error	$\frac{1}{\sigma^2}\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)\mathbf{x}_i$
logistic regression	Bernoulli	cross-entropy	$\sum_i (y_i - \sigma(\mathbf{w}^\top \mathbf{x}_i))\mathbf{x}_i$

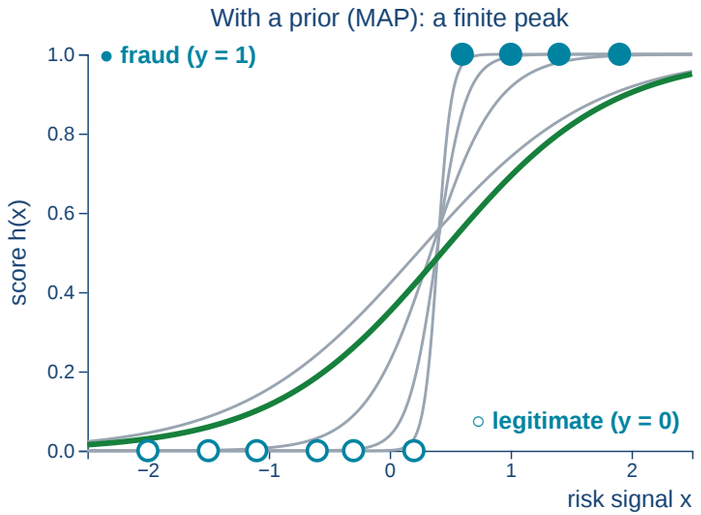
Error × feature — the LMS update with a sigmoid in the prediction; no closed form, so iterate. A fraud scored 0.01 costs $-\ln 0.01 \approx 4.6$, scored 0.99 only 0.01: confident mistakes dominate.

Speaker notes

Derivation for one row, using $\sigma'(z) = \sigma(z)(1-\sigma(z))$: $\frac{\partial}{\partial \mathbf{w}}[y \ln h + (1-y)\ln(1-h)] = \left(\frac{y}{h} - \frac{1-y}{1-h}\right) h(1-h)\,\mathbf{x} = (y-h)\,\mathbf{x}$. Both gradients in the table are error × feature; the per-example ascent step $\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y-h)\,\mathbf{x}$ is the LMS rule with the prediction $\sigma(\mathbf{w}^\top \mathbf{x})$.

Setting the logistic gradient to zero gives equations that are non-linear in $\mathbf{w}$, so there is no closed form. The negative log-likelihood is convex, so gradient methods or Newton's method (iteratively reweighted least squares) find the global optimum whenever one exists. Per-example loss: a fraud scored 0.01 costs $-\ln 0.01 \approx 4.61$, scored 0.99 costs $-\ln 0.99 \approx 0.01$; the cost grows without bound as a confident score approaches the wrong label.

When the Peak Runs Away



steps	slope	log-lik
10	1.4	-2.68
100	3.6	-1.04
1,000	8.3	-0.36
10,000	17.8	-0.06
MAP	**1.4**	**-2.54**

prior on the slope: N(0, 1)
stays, however long it runs

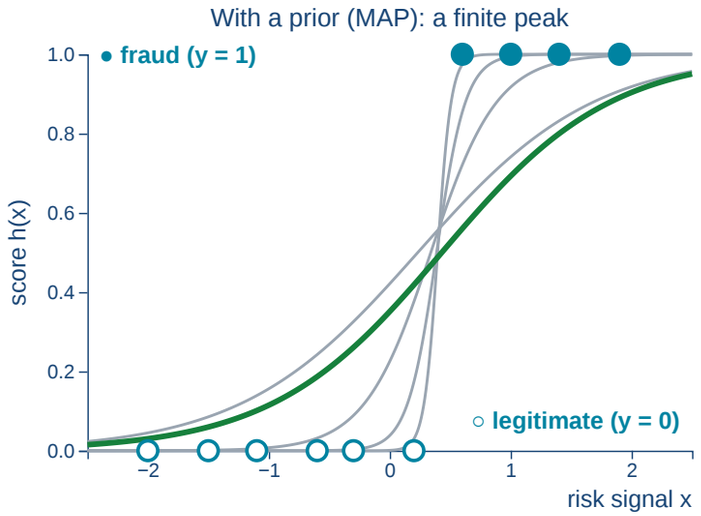
Every fraud scores above every legitimate case. Where is the peak of the likelihood?

Speaker notes

Data: one risk signal, six legitimate cases at $x \leq 0.2$ and four frauds at $x \geq 0.6$, perfectly separated. Gradient ascent with step size 0.05 from $\mathbf{w} = 0$; steps 1–4 show the curve after 10, 100, 1,000 and 10,000 steps, with slope 1.4, 3.6, 8.3, 17.8 and log-likelihood -2.68, -1.04, -0.36, -0.06. The log-likelihood is bounded above by 0 and approaches it only as the slope tends to infinity, so no finite maximiser exists; the optimiser behaves correctly. Step 5: a Gaussian prior $\mathcal{N}(0, 1)$ on the slope (intercept unpenalised); the MAP estimate converges to slope 1.4 and stays there.

The same holds in any dimension: if a hyperplane separates the classes, scaling its weights up raises the likelihood indefinitely. n points in general position are always separable when $n \leq p + 1$, so separation is common with many features and few positives. Libraries usually penalise by default — scikit-learn's LogisticRegression applies an L2 penalty unless told otherwise; Lecture 04 reads that penalty as a prior.

When the Peak Runs Away



steps	slope	log-lik
10	1.4	-2.68
100	3.6	-1.04
1,000	8.3	-0.36
10,000	17.8	-0.06
MAP	**1.4**	**-2.54**

prior on the slope: N(0, 1)
stays, however long it runs

Every fraud scores above every legitimate case. Where is the peak of the likelihood?

Nowhere. The likelihood keeps rising as the slope grows; the scores harden to 0 and 1 — three heads in three flips. A prior's peak (MAP) is finite.

Shortcut 3: Model Only $p(y \mid \mathbf{x})$

Linear and logistic regression are **discriminative**: a model of y given $\mathbf{x}$, none of $\mathbf{x}$.

What is dropped: $p(\mathbf{x})$

- a transaction unlike any seen still gets a confident score
- a missing field cannot be averaged out — it must be filled in
- no way to simulate a plausible transaction

What is gained

- no assumptions about how features are distributed — where, with hundreds of features, most modelling errors hide
- every parameter serves the decision

Lecture 03 took the other route: model the joint $p(\mathbf{x}, y)$ and condition — **generative**. Generative vs. Discriminative Models compares the two.

Speaker notes

A discriminative model specifies $p(y \mid \mathbf{x})$ and nothing about $p(\mathbf{x})$. Consequences: an input unlike any training example still receives a score, with no indication that it is unusual — the classification counterpart of the villa; a missing attribute cannot be marginalised out, since that requires a distribution for it given the observed ones; and the model cannot generate examples.

Gains: no assumptions about how the features are distributed or how they depend on each other, which is where a generative model with many features makes most of its modelling errors; and all parameters are fitted to the conditional likelihood, the quantity a decision uses. Given enough data, discriminative classifiers usually reach lower error than generative ones whose input model is misspecified (Lecture 04). The roadmap names this shortcut "only what you need".

Mini-Exercise

Your fraud model scores **0.97** for a transfer 50 times larger than any in its training data.

Should you trust the 0.97 — and could the model itself warn you?



Speaker notes

The 0.97 should not be taken at face value. The log-odds are linear in the amount, so a transfer 50 times larger than any seen pushes the score towards 0 or 1 by extrapolation, not by evidence; no training data supports the shape of the model in that region. The model cannot warn: detecting an unusual input needs a model of $p(\mathbf{x})$ — a density estimate, a range check on the inputs, or a generative classifier. The uncertainty involved is epistemic (Lecture 01): it comes from missing data in this region of input space, and training examples with similar amounts would reduce it.

Mini-Exercise

Your fraud model scores **0.97** for a transfer 50 times larger than any in its training data.

Should you trust the 0.97 — and could the model itself warn you?

Not blindly: the linear log-odds simply **extrapolate**, like the line at the villa. And no — the model has **no** $p(\mathbf{x})$, so nothing in it can see that the amount is unusual. That needs a model of $\mathbf{x}$ or a separate check.

Takeaway

Logistic regression is probabilistic linear regression with Bernoulli noise: linear log-odds, cross-entropy as the negative log-likelihood, error × feature as the gradient. It **keeps the peak** — which runs away on separable data — and **models only** $p(y \mid \mathbf{x})$: it never learns what a normal transaction looks like.

Generative vs. Discriminative Models

Guiding Question: Naive Bayes and logistic regression see the same fraud table. Which would you trust with 50 labelled frauds — and which with 50,000?

The Same Line, Fitted Twice

Take the logarithm of the naive-Bayes odds. For yes/no flags $x_j \in \{0, 1\}$ with $\theta_{jy} = P(x_j = 1 \mid y)$:

$$\log \frac{P(\text{fraud} \mid x)}{P(\text{honest} \mid x)} = w_0 + \sum_{j=1}^{p} w_j x_j, \qquad w_j = \log \frac{\theta_{j1}}{\theta_{j0}} - \log \frac{1 - \theta_{j1}}{1 - \theta_{j0}}$$

Where have you seen a log-odds that is linear in x?

Speaker notes

Derivation: for a yes/no flag, $\log p(x_j \mid y) = x_j \log \theta_{jy} + (1 - x_j) \log(1 - \theta_{jy}) = \log(1 - \theta_{jy}) + x_j \log \frac{\theta_{jy}}{1-\theta_{jy}}$. The naive-Bayes log-odds is the log prior odds plus the difference of these terms between $y = 1$ and $y = 0$. The constants collect in $w_0 = \log \frac{P(\text{fraud})}{P(\text{honest})} + \sum_j \log \frac{1-\theta_{j1}}{1-\theta_{j0}}$; the coefficients of x_j are the w_j on the slide.

New device, $\theta_{j1} = 0.6$, $\theta_{j0} = 0.1$: $w_j = \log 6 - \log(0.4/0.9) \approx 1.79 + 0.81 = 2.60$. The weight exceeds $\log 6$ because the absence of the flag is also evidence: honest transactions lack it more often. Naive Bayes estimates each θ_{jy} by counting, attribute by attribute; logistic regression maximises the conditional likelihood over all weights jointly. When two flags are correlated — new device and foreign IP arrive together in an account take-over — logistic regression splits the credit between them, while naive Bayes gives each its full weight: the double counting of Lecture 03.

The Same Line, Fitted Twice

Take the logarithm of the naive-Bayes odds. For yes/no flags $x_j \in \{0, 1\}$ with $\theta_{jy} = P(x_j = 1 \mid y)$:

$$\log \frac{P(\text{fraud} \mid x)}{P(\text{honest} \mid x)} = w_0 + \sum_{j=1}^{p} w_j x_j, \qquad w_j = \log \frac{\theta_{j1}}{\theta_{j0}} - \log \frac{1 - \theta_{j1}}{1 - \theta_{j0}}$$

Where have you seen a log-odds that is linear in x?

> **Logistic regression** (previous module): $P(\text{fraud} \mid x) = \sigma(w_0 + w^\top x)$. Same family of classifiers — naive Bayes sets the weights by **counting**, one attribute at a time; logistic regression by **maximising** $\sum_i \log p(y_i \mid x_i)$, all weights jointly.

Two Ways to a Classifier

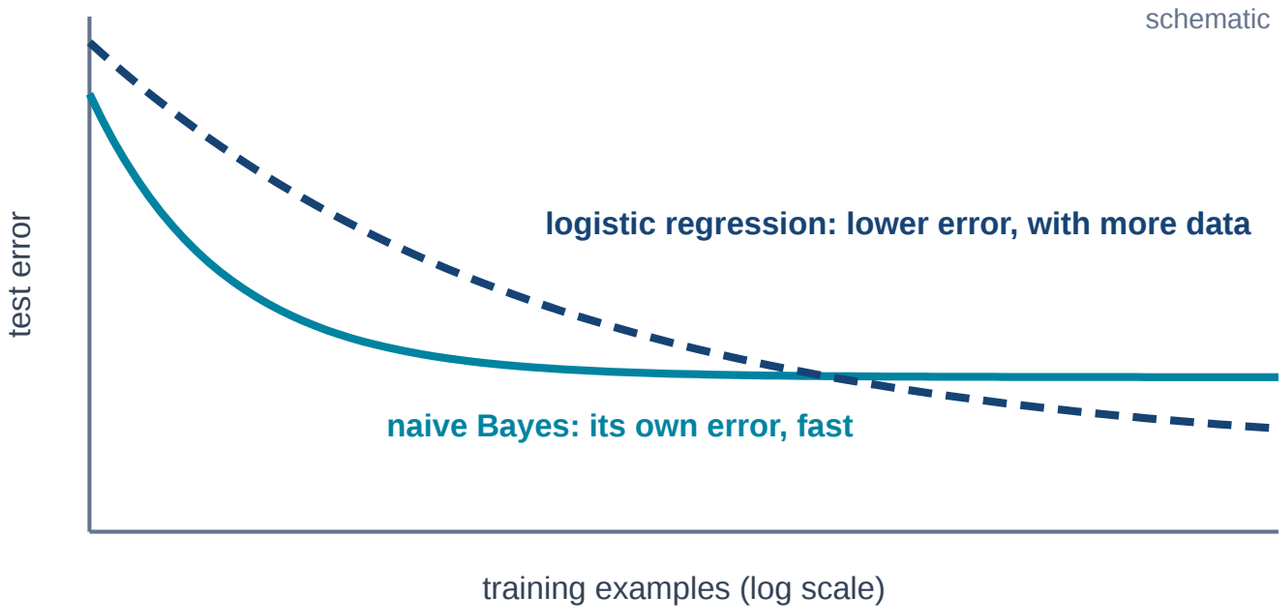
	naive Bayes — generative	logistic regression — discriminative
models	$P(y)$ and $p(x \mid y)$	$P(y \mid x)$ only
fitted by	counting (joint likelihood)	iterative optimisation (conditional likelihood)
correlated flags	counted twice	credit shared
a missing field	drop its factor	needs a value
an unusual x	low $p(x)$: can be flagged	still a confident score
a new base rate	change $P(y)$	refit or correct the intercept
drops	dependences between attributes	everything about x

Speaker notes

Fitting: naive Bayes maximises the joint likelihood $\prod_i P(y_i)\, p(\mathbf{x}_i \mid y_i)$, whose maximiser is a set of relative frequencies; logistic regression maximises the conditional likelihood $\prod_i P(y_i \mid \mathbf{x}_i)$, which has no closed-form maximiser. Missing field: in a product of per-attribute factors the missing factor sums to 1 over its values and drops out. Unusual $\mathbf{x}$: naive Bayes yields $p(\mathbf{x}) = \sum_y P(y)\, p(\mathbf{x} \mid y)$, so a low value flags an input unlike the training data — judged under the independence assumption, which can misjudge how unusual a combination is.

New base rate: in naive Bayes only $P(y)$ changes. In logistic regression the base rate is part of w_0; a change of prior odds from o to o' shifts w_0 by $\log(o'/o)$, valid if $p(\mathbf{x} \mid y)$ is unchanged. Last row: each model is characterised by what it leaves out.

Few Data, Many Data



Few labelled frauds: naive Bayes. Its assumption acts like a strong prior — little to estimate, so it settles fast.
Many labelled frauds: logistic regression. It pays for fewer assumptions with data, and ends lower.

Shown empirically and in theory by Ng & Jordan (2002). A wrong assumption costs accuracy in the limit and buys it early — the prior of Lecture 03, again.

Speaker notes

The figure is schematic, not measured. Ng & Jordan (NIPS 2002, "On discriminative vs. generative classifiers") compare naive Bayes with logistic regression: naive Bayes approaches its asymptotic error with a number of examples logarithmic in the number of attributes, logistic regression needs a number linear in it, and the asymptotic error of logistic regression is never higher. On many real datasets the learning curves cross.

Answer to the module's question: with 50 labelled frauds naive Bayes is the safer choice, with 50,000 logistic regression. An assumption is information: when roughly right it substitutes for data, when wrong it limits the error that any amount of data can reach.

Mini-Exercise

Our three-flag transaction arrives **without the IP field** — the logging failed.

What does naive Bayes do with it — and what does logistic regression do?

Speaker notes

Naive Bayes marginalises the missing attribute: for each class, $\sum_v p(x_{\text{IP}} = v \mid y) = 1$, so the IP factor disappears and the likelihood ratios 6 (new device) and 2.5 (night) remain. Odds $\frac{1}{999} \cdot 6 \cdot 2.5 = 15/999$, probability $15/1014 \approx 1.5\%$. With a foreign IP the odds were $210/999$, i.e. 17.4%; with a domestic IP the ratio is $0.3/0.95 \approx 0.32$ and the probability about 0.5%. The 1.5% is the average of the two cases, weighted by how likely each IP value is given the other flags: the missing value is accounted for, not guessed.

Logistic regression has a weight for x_{IP} but no distribution for it. It needs an imputed value, which commits to a guess, or a second model trained without the field.

Mini-Exercise

Our three-flag transaction arrives **without the IP field** — the logging failed.

What does naive Bayes do with it — and what does logistic regression do?

Naive Bayes drops the factor: $\sum_v p(x_{\text{IP}} = v \mid y) = 1$, so the odds become $6 \cdot 2.5 : 999$ — about **1.5%**. **Logistic regression** has no model of x: it needs *some* value for x_{IP}, so someone must impute one, or train a second model without the field.

Takeaway

> Naive Bayes and logistic regression draw **the same kind of line**; they differ in what they model. The generative model pays with assumptions about x and gets answers about x; the discriminative one pays with data.

lecture	shortcut	what is dropped
Lecture 03	keep the peak	how sure we are
Lecture 03	assume it apart	every dependence between attributes
Lecture 04	only $p(y \mid x)$	every question about x

Naive Bayes is a graph with one hub. **Lecture 05** draws the others — and keeps exactly the dependences we claim.

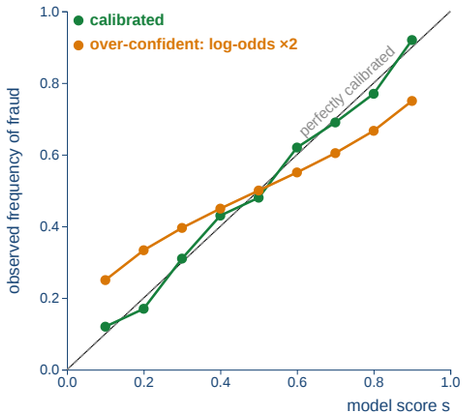
Speaker notes

The table lists the three shortcuts from the full Bayesian treatment so far. Keep the peak replaces the posterior over parameters by its maximum; assume it apart replaces the joint distribution of the attributes by a product of per-attribute factors; only $p(y \mid x)$ replaces the joint $p(x, y)$ by the conditional. Naive Bayes is the simplest directed graphical model: a class node with an arrow to each attribute and no arrows between attributes. Graphical models (Lecture 05) allow any chosen set of arrows, so each independence assumption becomes an explicit statement about specific variables rather than a blanket assumption over all attribute pairs.

Calibration

Guiding Question: Lecture 01 promised to say when a model's output earns the name "probability". When does it — and what breaks it?

When Is 0.7 a Probability?



A score is **calibrated** if, among all cases scored s, a fraction s is positive. A **reliability diagram** plots that fraction against s.

Predict before click 2: a model whose log-odds are twice too large — above or below the diagonal at high scores?

Speaker notes

Step 0: the diagonal, where score equals observed frequency. Step 1: a calibrated model; its points scatter around the diagonal because each bin holds finitely many cases. Step 2: a model whose log-odds are twice the correct ones. Its scores are too extreme: below the diagonal at high scores, above it at low ones. A reported 0.9 has log-odds $\ln 9 \approx 2.20$; halving gives $\ln 3 \approx 1.10$, a true frequency of 0.75. Symmetrically, 0.1 is really 0.25.

Naive Bayes is over-confident in this way when attributes are correlated, because correlated evidence is counted twice (Lecture 03). A cost-based threshold such as $p^* = 0.1$ in Lecture 01 gives the cost-optimal decision only if the score is the actual probability. Calibration and ranking are separate properties: a model can rank perfectly and be badly calibrated, or be calibrated and rank poorly.

What Maximum Likelihood Guarantees

At the peak, the gradient for the intercept ($x_0 = 1$) is zero:

$$\sum_{i=1}^{n}(y_i - h_i) = 0 \quad \Longrightarrow \quad \frac{1}{n}\sum_{i=1}^{n} h_i = \frac{1}{n}\sum_{i=1}^{n} y_i$$

What does this equation promise — and what does it not?

Speaker notes

The equation is the intercept component of the gradient $\sum_i (y_i - h_i)\mathbf{x}_i$, which vanishes at the maximum. It requires an unpenalised intercept and an attained maximum. For a yes/no feature the same argument applied to its weight gives a second guarantee: among the training cases with that feature, the mean score equals the fraction of positives.

Per-score calibration needs a correctly specified model — log-odds truly linear in the features — which is rarely exact; logistic regression is often close in practice, which explains its reputation for calibrated probabilities. All guarantees refer to the training distribution; a changed base rate or rebalanced training breaks them. A constant score equal to the base rate is perfectly calibrated and useless for separating the classes. Separation — ranking frauds above legitimate cases — is measured in Lecture 06.

What Maximum Likelihood Guarantees

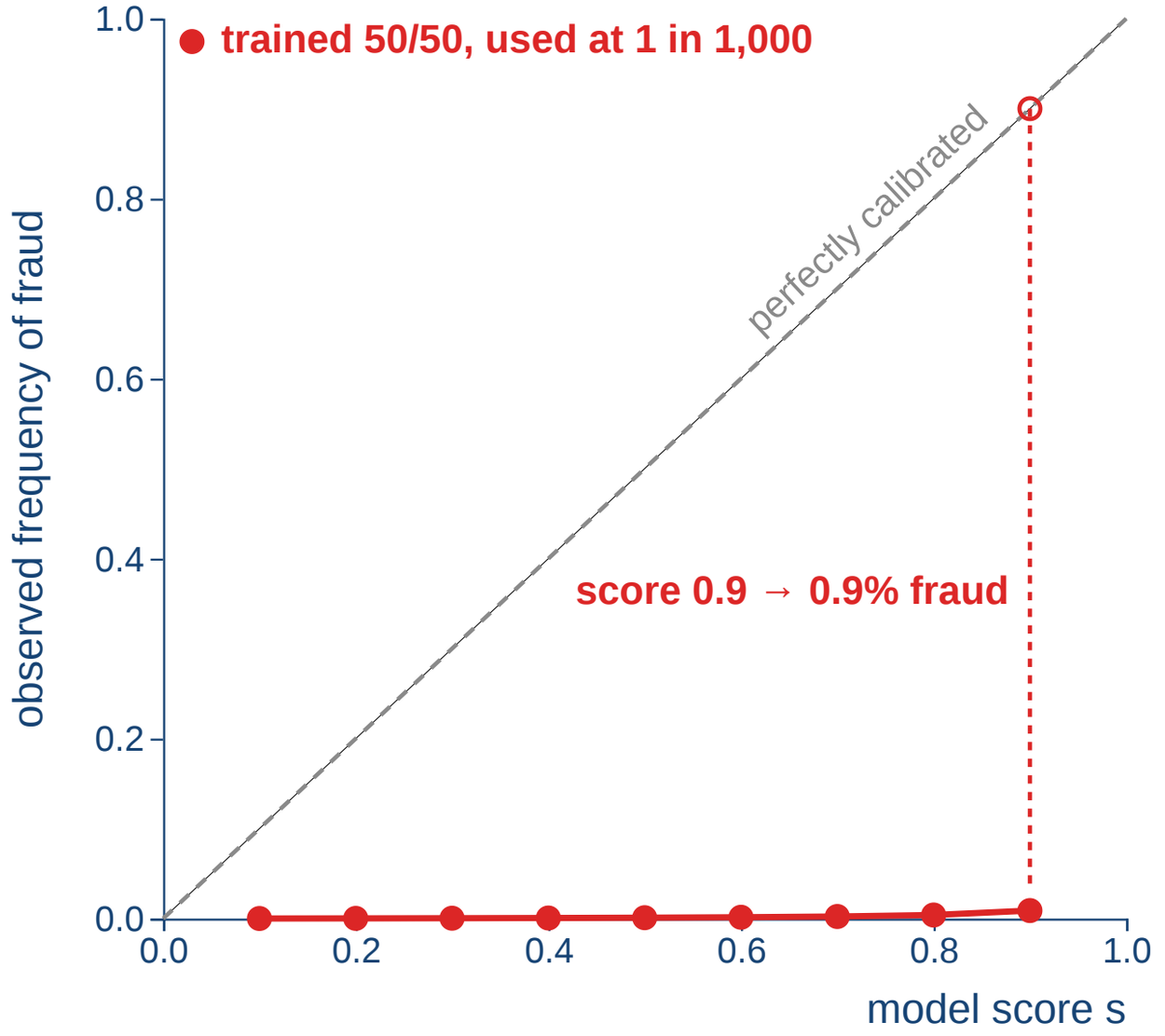
At the peak, the gradient for the intercept ($x_0 = 1$) is zero:

$$\sum_{i=1}^{n}(y_i - h_i) = 0 \quad \Longrightarrow \quad \frac{1}{n}\sum_{i=1}^{n} h_i = \frac{1}{n}\sum_{i=1}^{n} y_i$$

What does this equation promise — and what does it not?

promised	not promised
average score = training fraud rate: *calibration in the large*	each score right — needs log-odds truly linear in $\mathbf{x}$
on the **training** distribution	after a base-rate shift, or **rebalancing**
	that it **separates**: the base rate for everyone is calibrated — and useless

Rebalanced Training



Fraud is 1 in 1,000, so practitioners **undersample** legitimate cases to 50/50. The model learns odds for a world where fraud is common.

Bayes in odds form:

$$\text{real odds} = \text{model odds} \times \frac{1/999}{1}$$

Score 0.9: odds $9 \to 9/999$, about **0.9%**.



Speaker notes

On real traffic the rebalanced model's points lie close to the floor: a score of 0.9 corresponds to a fraud frequency of about 0.9% (odds $9/999$, probability $9/1008 \approx 0.89\%$).

Why the correction works: undersampling legitimate cases changes the class prior but not $p(\mathbf{x} \mid y)$, provided legitimate cases are dropped at random. The model's odds are therefore the likelihood ratio times the training prior odds, and the real odds are obtained by exchanging the prior: real odds = model odds × real prior odds / training prior odds, here $(1/999)/1$. This is Bayes' theorem in odds form (Lecture 02), and the same base-rate effect that made a "99% accurate" flag mean 9% there. A threshold of 0.5 on the raw score corresponds to a real fraud probability of 0.1%, so almost every case it blocks is legitimate.

Regularisation Is a Prior

$$\min_{\mathbf{w}} \mathrm{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$$

You know this as ridge regression or weight decay. Which prior's peak is it?



Speaker notes

Multiplying the log-posterior by $-2\sigma^2$ gives $\mathrm{RSS} + \frac{\sigma^2}{\tau^2}\|\mathbf{w}\|^2$, so $\lambda = \sigma^2/\tau^2$: noisy data (large σ^2) or a tight prior (small τ^2) means more shrinkage, and $\tau^2 \to \infty$ recovers least squares. This is the Beta(2, 2) exercise of Lecture 03 one level up: there the prior pulled the peak from 0.8 to 0.75, here the Gaussian prior pulls the weights towards 0. For logistic regression the same prior gives cross-entropy $+\|\mathbf{w}\|^2/(2\tau^2)$ — the finite peak of the runaway example. The intercept is usually left unpenalised, so calibration in the large survives.

L1: the Laplace log-density $-|w|/b$ has a corner at 0. At a corner the peak can sit exactly at 0 for a whole range of data, which produces exact zeros. The posterior mean averages over the posterior and is never exactly 0.

Regularisation Is a Prior

$$\min_{\mathbf{w}} \mathrm{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$$

You know this as ridge regression or weight decay. Which prior's peak is it?

MAP from Lecture 03 with $\mathbf{w} \sim \mathcal{N}(0, \tau^2 \mathbf{I})$: $\log p(\mathbf{w} \mid D) = -\frac{\mathrm{RSS}}{2\sigma^2} - \frac{\|\mathbf{w}\|^2}{2\tau^2} + \mathrm{const}$, so $\lambda = \sigma^2/\tau^2$.

penalty	prior on $\mathbf{w}$	what its peak does
$\lambda \|\mathbf{w}\|^2$	Gaussian	shrinks; keeps separable data finite
$\lambda \|\mathbf{w}\|_1$	Laplace	exact zeros — a property of the **peak** (the kink at 0); the posterior mean has none

Mini-Exercise

A fraud model trained on a **50/50** sample is used on real traffic, where fraud is **1 in 1,000**. It outputs **0.5** and **0.8**.

What are the real fraud probabilities?

Speaker notes

Rule: multiply the model odds by $1/999$. Score 0.5: odds 1, real odds $1/999$, probability $1/1000 = 0.100\%$ — the base rate itself. Score 0.8: odds 4, real odds $4/999$, probability $4/1003 \approx 0.399\%$. A score of 0.5 on a balanced training sample means "nothing unusual" on real traffic: the model's evidence is neutral, and the base rate decides.

Mini-Exercise

A fraud model trained on a **50/50** sample is used on real traffic, where fraud is **1 in 1,000**. It outputs **0.5** and **0.8**.

What are the real fraud probabilities?

Multiply the odds by $1/999$. Score 0.5: odds $1 \to 1/999$, about **0.1% —** the base rate itself. Score 0.8: odds $4 \to 4/999$, about **0.4%**.

Takeaway

A score is a probability when it is **calibrated** — checkable on a reliability diagram, not by accuracy. Maximum likelihood guarantees it only on average and only on the training distribution; **rebalancing** breaks it, and the prior odds repair it. **Regularisation is a prior**, and a penalised fit is a MAP estimate.

Every Loss So Far Is a Likelihood

So far — Lecture 06 meets losses that are not.

you minimised	it is $-\log$ of	the fit is
squared error	Gaussian noise around $\mathbf{w}^\top \mathbf{x}$	the peak (MLE)
cross-entropy	Bernoulli labels with bias $\sigma(\mathbf{w}^\top \mathbf{x})$	the peak (MLE)
$+\lambda \lVert \mathbf{w} \rVert^2$	a Gaussian prior on $\mathbf{w}$	a prior's peak (MAP)
$+\lambda \lVert \mathbf{w} \rVert_1$	a Laplace prior on $\mathbf{w}$	a prior's peak (MAP)

Dropped: the spread

how sure we are about $\mathbf{w}$ — **Lecture 07** keeps it

Dropped: $p(\mathbf{x})$

what a normal input looks like — **Lecture 03** modelled it, naive Bayes

Speaker notes

Each row identifies an objective with a probability model. Minimising squared error maximises a Gaussian likelihood of fixed variance; minimising cross-entropy maximises a Bernoulli likelihood; adding $\lambda\|\mathbf{w}\|^2$ adds $-\log$ of a Gaussian prior $\mathcal{N}(0, \tau^2\mathbf{I})$, with $\lambda = \sigma^2/\tau^2$ in regression; adding $\lambda\|\mathbf{w}\|_1$ adds $-\log$ of a Laplace prior. In every row the result is a single point: the peak of the likelihood (MLE) or of the posterior (MAP).

"So far" is literal. Not every loss is a negative log-likelihood: the hinge loss of support vector machines is not $-\log p(y \mid \mathbf{x})$ of any model, because $e^{-\text{loss}}$ summed over the two labels depends on the score and cannot be normalised without changing the minimiser. Minimising such a loss directly, with no probability model behind it, is the shortcut of Lecture 06.

The cards name what the two shortcuts of this lecture leave out. The spread of $\mathbf{w}$ is what widens the band at the villa from ±31 k€ to ±153 k€; Lecture 07 keeps it. $p(\mathbf{x})$ is what would flag a transfer unlike any seen; naive Bayes (Lecture 03) models it.